

Limits & Convergence of Algorithms

Growth rates revisited, sequences, and the order of convergence

Sharareh Sayyad

Washington state University

September 30, 2026

Outline

Review: polynomial vs. exponential growth

Limits of sequences

Algorithms as sequences

Order of convergence

Summary

Review: polynomial vs. exponential growth

Polynomial: $O(n^a)$

- ▶ nested loops: $n, n^2, n^3, \dots$
- ▶ **double** $n \Rightarrow \text{work} \times 2^a$

Exponential: $O(b^n), b > 1$

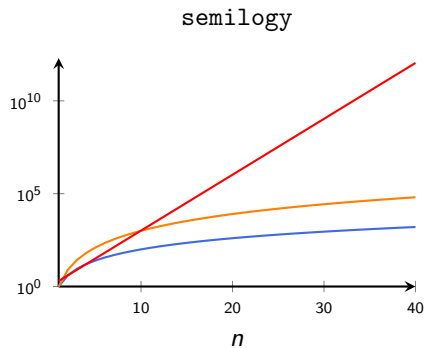
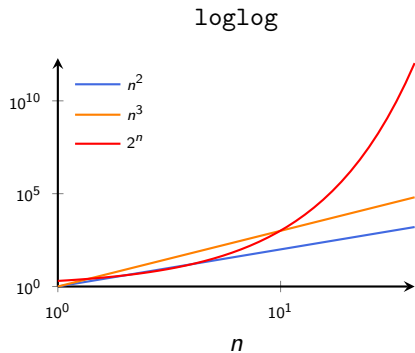
- ▶ try every subset: 2^n
- ▶ **add one** to $n \Rightarrow \text{work} \times b$

n	n^3	2^n
10	1 000	1 024
20	8 000	1.0×10^6
40	64 000	1.1×10^{12}
80	512 000	1.2×10^{24}

Already a limit!

For any a and any $b > 1$: $\lim_{n \rightarrow \infty} \frac{n^a}{b^n} = 0$.
Every exponential eventually beats every polynomial.

Which axes make it a straight line?



growth	take logs	straight on	slope
$t \approx c n^a$	$\log t \approx a \log n + \log c$	loglog	a (exponent)
$t \approx c b^n$	$\log t \approx n \log b + \log c$	semilogy	$\log b$ ($b = e^{\text{slope}}$)

Recipe: fit the slope, then test the model

```
# polynomial model: slope = a
a = np.polyfit(np.log(ns),
               np.log(ts), 1)[0]

# exponential model: slope = log b
s = np.polyfit(ns, np.log(ts), 1)[0]
b = np.exp(s)
```

`polyfit` **always** returns a number, even for the wrong model. So how do we know which model is right?

Reminder: fit only the large- n part, as last time.

The split test

Fit the model twice: on the **first half** of the data, then on the **second half**. Right model: both fits **agree**. Wrong model: the value **drifts**.

Example. Data $t = n^3$, for $n = 10, 12, \dots, 40$:

model	first half $n = 10 \dots 24$	second half $n = 26 \dots 40$	verdict
polynomial, a	3.00	3.00	same
exponential, b	1.20	1.10	drifts

So the data is polynomial, $O(n^3)$, not exponential.

Sequences and limits

A **sequence** is a list of numbers $x_0, x_1, x_2, \dots$ indexed by k .

Definition

$x_k \rightarrow x_*$ ("converges to x_* ") if for every tolerance $\tau > 0$ there is a K with

$$|x_k - x_*| < \tau \quad \text{for all } k \geq K.$$

In words: choose any tolerance τ , however small. After some index K , **every** term stays within τ of x_* .

Example: $x_k = 1/k \rightarrow 0$

Tolerance $\tau = 0.01$: take $K = 101$. Every $k \geq 101$ gives $1/k < 0.01$.

Smaller τ just needs a larger K .

sequence	behaviour
$1/k$	$\rightarrow 0$
$(1 + 1/k)^k$	$\rightarrow e$
$\sum_{j=1}^k 1/j^2$	$\rightarrow \pi^2/6$
$(-1)^k$	no limit (oscillates)
k^2	$\rightarrow \infty$ (diverges)

Converging means the terms settle down to **one** finite number.

An algorithm's output is a sequence

Record the output after every iteration:

$$x_0 \xrightarrow{g} x_1 \xrightarrow{g} x_2 \xrightarrow{g} \cdots, \quad x_{k+1} = g(x_k).$$

algorithm	x_k
Newton for $\sqrt{2}$	$x_{k+1} = \frac{1}{2}(x_k + 2/x_k)$
cos iteration	$x_{k+1} = \cos x_k$
partial sums of a series	$S_{k+1} = S_k + a_{k+1}$

Where does it converge?

If $x_k \rightarrow x_*$ and g is continuous, take the limit on both sides of $x_{k+1} = g(x_k)$:

$$x_* = g(x_*).$$

The limit must be a **fixed point** of g .

So: **solve** $x = g(x)$ **to find the candidates**, then ask which one (if any) the iteration actually reaches.

$$\text{Newton: } x = \frac{1}{2}(x + 2/x) \iff x^2 = 2.$$

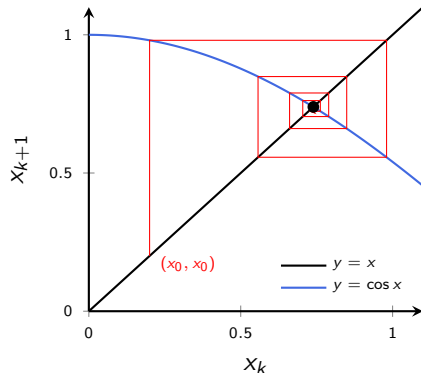
Seeing it: the cobweb plot

Why draw it? The numbers $x_0, x_1, \dots$ tell you *whether* the iteration settles; the cobweb shows *where* (where $y = g(x)$ crosses $y = x$) and *how* (spirals in, walks in, or runs away).

How: each iteration draws **two** line segments:

1. **vertical** to the curve: $(x_k, x_k) \rightarrow (x_k, x_{k+1})$;
2. **horizontal** back to $y = x$:
 $(x_k, x_{k+1}) \rightarrow (x_{k+1}, x_{k+1})$.

```
x = np.linspace(0, 1.1, 200)
plt.plot(x, np.cos(x))      # curve  $y = g(x)$ 
plt.plot(x, x, "k")         # line  $y = x$ 
xk = 0.2                    # start  $x_0$ 
for k in range(10):
    xnew = np.cos(xk)        #  $x_{k+1} = g(x_k)$ 
    plt.plot([xk, xk], [xk, xnew], "r")
    plt.plot([xk, xnew], [xnew, xnew], "r")
    xk = xnew
```



$x_{k+1} = \cos x_k$ from $x_0 = 0.2$ spirals in to the crossing $x_* \approx 0.739$.

Which fixed point does the iteration find?

The problem

g can have **several** fixed points, but the iteration can end at **at most one**, or at none. Which one, and why?

Example: $g(x) = x^2$

Fixed points solve $g(x_*) = x_*$: $x_*^2 = x_*$, so $x_* = 0$ or $x_* = 1$.

$x_0 = 0.9$: 0.81, 0.66, 0.43, ... $\rightarrow 0$

$x_0 = 1.1$: 1.21, 1.46, 2.14, ... $\rightarrow \infty$

Starting close to 1 does not help: the iterates move **away** from 1.

Slopes: $g'(x) = 2x$, so $g'(0) = 0$ and $g'(1) = 2$.

The reason: the slope of g at x_* . Near x_* , one step multiplies the error by about $g'(x_*)$:

$$x_{k+1} - x_* = g(x_k) - g(x_*) \approx g'(x_*)(x_k - x_*).$$

Local test

- ▶ $|g'(x_*)| < 1$: error shrinks, x_* **attracts**, with rate $C = |g'(x_*)|$;
- ▶ $|g'(x_*)| > 1$: error grows, x_* **repels**.

$g(x) = x^2$: 0 attracts, 1 repels.

$g(x) = \cos x$, $g'(x) = -\sin x$: at $x_* \approx 0.739$, $|g'(x_*)| = \sin(0.739) \approx 0.67 < 1$: attracts, error $\times 0.67$ per step.

Order of convergence

Error $\epsilon_k = |x_k - x_*|$. The key question: **how does one iteration shrink the error?**

Definition

The convergence has **order** $q \geq 1$ and **rate constant** C if

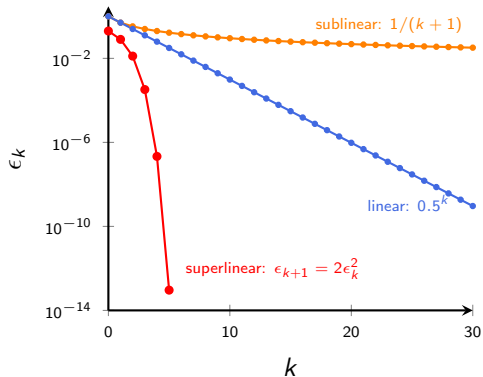
$$\lim_{k \rightarrow \infty} \frac{\epsilon_{k+1}}{\epsilon_k^q} = C, \quad 0 < C < \infty.$$

For $q = 1$: $C < 1$ is **linear**, $C = 1$ is **sublinear**.

name	ratio $\epsilon_{k+1}/\epsilon_k$
sublinear	not constant: it creeps up to 1 e.g. $\epsilon_k = 1/k$: $\frac{1}{2}, \frac{2}{3}, \frac{3}{4}, \dots \rightarrow 1$
linear	constant $C < 1$
superlinear	goes to 0: each step helps more than the last

Sublinear, linear, superlinear on a semilogy plot

type	on semilogy
sublinear	flattens out: each step gains less
linear	straight line, slope $\log C$
superlinear	bends down: each step gains more



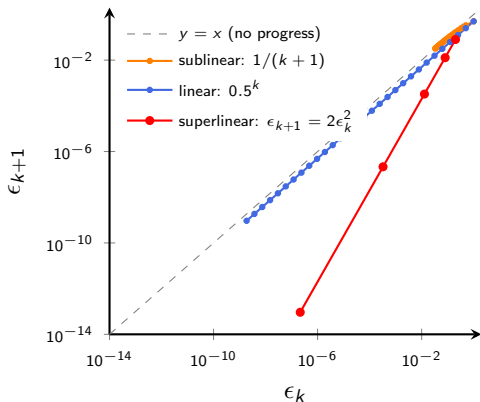
Measuring q : plot ϵ_{k+1} against ϵ_k

Take logs of $\epsilon_{k+1} \approx C \epsilon_k^q$:

$$\log \epsilon_{k+1} \approx q \log \epsilon_k + \log C.$$

A **power law** again, so on loglog axes the points $(\epsilon_k, \epsilon_{k+1})$ lie on a line with **slope q** .

- ▶ **sublinear**: slope 1, hugging $y = x$ ($C = 1$: almost no progress);
- ▶ **linear**: slope 1, parallel **below** $y = x$ (by the factor $C = 0.5$);
- ▶ **superlinear**: **steeper**, here slope 2.



the same three sequences as on the previous slide; the steeper the line, the higher the order

Measuring q : three consecutive errors

Divide $\epsilon_{k+1} \approx C\epsilon_k^q$ by $\epsilon_k \approx C\epsilon_{k-1}^q$: C cancels, and

$$q_k = \frac{\log(\epsilon_{k+1}/\epsilon_k)}{\log(\epsilon_k/\epsilon_{k-1})}.$$

```
def order_estimates(eps):  
    r = np.log(eps[1:] / eps[:-1])  
    return r[1:] / r[:-1]
```

One estimate per k : the early ones are rough, so watch them **settle**.

Example: Newton

Errors 3.3×10^{-2} , 5.8×10^{-4} , 1.9×10^{-7} :

$$q \approx \frac{\log(1.9 \times 10^{-7}/5.8 \times 10^{-4})}{\log(5.8 \times 10^{-4}/3.3 \times 10^{-2})} = \frac{-8.0}{-4.0} \approx 2.$$

Floating point floor

Use only errors well above 10^{-16} (e.g. $\epsilon_k > 10^{-14}$). At the floor the ratios are rounding noise, and $\log 0 = -\infty$.

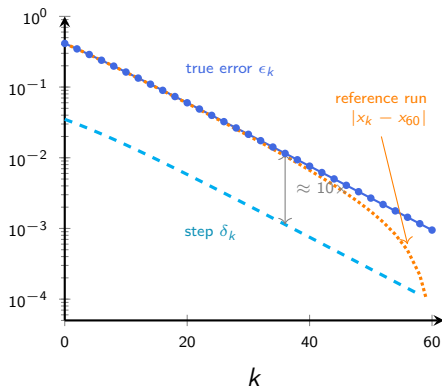
What if x_* is not known?

Usually we do **not** know the answer. Two substitutes:

1. **Reference run:** use the last iterate x_N as x_* . Matches the error, except close to N .
2. **Steps** $\delta_k = |x_{k+1} - x_k|$: **parallel** to the error, so same slope, same C and q .

Careful when stopping

Parallel is not equal: the error is about $\frac{C}{1-C} \delta_k$, here $\approx 10 \delta_k$. Stop when $\delta_k < \text{tol}$, and always cap the iterations.



$x_{k+1} = x_k - 0.035(x_k^2 - 2)$, which converges slowly ($C \approx 0.9$) to $\sqrt{2}$

Summary

Growth and limits

- ▶ n^a : straight on loglog (slope a); b^n : straight on semilogy (slope $\log b$). Split test to pick the model.
- ▶ $x_k \rightarrow x_*$: after some K , all terms stay within any tolerance of x_* .
- ▶ Limits of $x_{k+1} = g(x_k)$ are fixed points $x_* = g(x_*)$; attracting if $|g'(x_*)| < 1$. Cobweb plots show it.

Order of convergence

- ▶ $\epsilon_{k+1} \approx C \epsilon_k^q$: sublinear (ratio $\rightarrow 1$), linear ($q = 1$, $C < 1$), superlinear ($q > 1$), quadratic ($q = 2$, digits double).
- ▶ semilogy of ϵ_k : flattens / straight / bends down.
- ▶ Measure q : slope of $\log \epsilon_{k+1}$ vs. $\log \epsilon_k$, or q_k from three errors; use errors above $\approx 10^{-14}$.
- ▶ x_* unknown: use a reference run or the steps δ_k .